UNIT 5 70 min

Loops: Repeating Work

for loops over ranges, while loops with conditions, accumulators, break, and continue.

By the end of this unit you will be able to

- Write a `for` loop over a `range()` and explain what the loop variable holds
- Write a `while` loop and identify the update that makes it end
- Use an accumulator to build up a total
- Use `break` and `continue` to change the flow of a loop
- Recognize and avoid an infinite loop

Why loops

A loop repeats a block of code. Without loops you would copy a line 1,000 times. With them, the program repeats the same logic for each item, each number, or until a condition changes.

for loops and range()

A `for` loop visits each item in a sequence, one at a time. `range(stop)` produces the numbers from 0 up to but **not including** `stop`. `range(start, stop)` starts at `start`. `range(start, stop, step)` counts by `step`.

for_range.py

```
for i in range(3):
    print("Count:", i)

for n in range(2, 10, 3):
    print("Stepped:", n)
```

Trace: summing 1 to 3 TRACE

```
total = 0
for i in range(1, 4):
    total = total + i
print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`total = 0`	`—`
2	2	`total = 0, i = 1`	`—`
3	3	`total = 1, i = 1`	`—`
4	2	`total = 1, i = 2`	`—`
5	3	`total = 3, i = 2`	`—`
6	2	`total = 3, i = 3`	`—`
7	3	`total = 6, i = 3`	`—`
8	2	`total = 6, i = 3`	`—`
9	4	`total = 6, i = 3`	`6`

Note

`range(1, 4)` gives 1, 2, 3. It does not include 4. That is the most common off-by-one mistake in this unit.

while loops

A `while` loop keeps running as long as its condition is True. It is checked before each pass. You must change something inside the loop, or it never stops.

countdown.py

```
count = 3
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```

Trace: the countdown TRACE

```
1  count = 3
2  while count > 0:
3      print(count)
4      count = count - 1
5  print("Liftoff!")
```

Step	Line run	Variables after the line	Output so far
1	1	`count = 3`	—
2	2	`count = 3`	—
3	3	`count = 3`	`3`
4	4	`count = 2`	`3`
5	2	`count = 2`	`3`
6	3	`count = 2`	`3 | 2`
7	4	`count = 1`	`3 | 2`
8	2	`count = 1`	`3 | 2`
9	3	`count = 1`	`3 | 2 | 1`
10	4	`count = 0`	`3 | 2 | 1`
11	2	`count = 0`	`3 | 2 | 1`
12	5	`count = 0`	`3 | 2 | 1 | Liftoff!`

Watch out

Infinite loops. If the condition never becomes False, the program runs forever. Common causes: forgetting to update the variable, or using `while True:` with no `break`. If you run one in this site, press the browser's reload button.

break and continue

- `break` leaves the loop immediately.
- `continue` skips the rest of this pass and moves to the next item.

break_continue.py

```
for n in range(1, 8):
    if n == 3:
        continue       # skip 3
    if n == 6:
        break          # stop before 6
    print(n)
```

Loops and conditions together

Loops and `if` statements combine freely. A common pattern is a loop that searches for something and stops when it finds it.

search.py

```
scores = [72, 95, 88, 61]
for s in scores:
    if s >= 90:
        print("First score at or above 90:", s)
        break
else:
    print("No score reached 90.")
```

Tip

The `else` on a loop runs only when the loop finished *without* hitting `break`. It is unusual, so label it clearly in your code.

Predict the output THINK FIRST

Trace each loop in your head before you run it.

```
for i in range(3):
    print(i)
```

Answer: `0 1 2`

`range(3)` stops before 3.

```
print(list(range(2, 10, 3)))
```

Answer: `[2, 5, 8]`

```
for i in range(5):
    if i == 2:
        break
    print(i)
```

Answer: `0 1`

```
for i in range(4):
    if i % 2 == 0:
        continue
    print(i)
```

Answer: `1 3`

```
n = 0
while n < 3:
    n = n + 1
    print(n)
```

Answer: `1 2 3`

```
print(sum(range(1, 5)))
```

Answer: `10`
1 + 2 + 3 + 4.

Practice

▼ **Exercise 1 — Divisible by three**

Print every number from 1 to 20 that is divisible by 3.

▼ **Hint**

Use `%`. A number is divisible by 3 when `n % 3 == 0`.

▼ **Show solution**

Solution

```
for n in range(1, 21):
    if n % 3 == 0:
        print(n)
```

▼ **Exercise 2 — Running total**

Starting at `total = 0`, add 7 to `total` each time, using a `while` loop. Stop as soon as `total` is 50 or more. Print the final total.

▼ **Show solution**

Solution

```
total = 0
while total < 50:
    total = total + 7
print(total)
```

Discussion

1. When would you choose `for` and when `while`? Give one example of each from everyday life.
2. The trace shows line 2 running one more time than the body. Why does the check need to run one extra time?
3. What is the risk of using `while True:`, and how does a `break` reduce it?

▼ **Facilitator notes**

Ask students to predict the final value of `total` in the running-total exercise before running it: 7, 14, ..., 56. The answer is 56, not 50, because the check is done before each addition.

Key terms

Loop

A statement that repeats a block of code.

Iteration

One pass through the body of a loop.

for loop

A loop that assigns each item of a sequence to a variable in turn.

while loop

A loop that repeats while its condition is True. The condition is checked before each pass.

range()

A built-in that produces a sequence of integers from `start` up to but not including `stop`.

Accumulator

A variable that collects a running result, such as a total, across loop iterations.

break

A statement that exits the nearest loop immediately.

continue

A statement that skips the rest of the current iteration and moves to the next.

Infinite loop

A loop whose condition never becomes False, so it never ends unless interrupted.

Off-by-one error

A bug caused by a range or index that is one too high or too low.

See all terms in the glossary →